

Coercive Subtyping via Logic Programming for Painless Mendler-style Algebraic Programming

No Author Given

No Institute Given

Abstract. The paper proposes coercive subtyping to infer combinators needed for algebraic programming.

Keywords: Strong functional programming, Mendler-style recursion, coercive subtyping

1 Introduction

Strong functional programming is a form of pure functional programming where all functions are statically required to terminate on all inputs. One way to achieve this is through an algebraic approach: datatypes are least fixed points μF of signature functors F , and recursive functions are least fixed points $\llbracket a \rrbracket$ of algebras a . The functions are called catamorphisms. The signature functor can be seen as describing one layer of the data. The least fixed point then describes data consisting of any finite number of layers. We may unroll μF to $F(\mu F)$, and roll it back again. Such transformations are needed to allow construction of data by applying constructors of the signature functor, and destruction by pattern-matching against those constructors. Algebras interpret the operations declared for one layer of the data, and catamorphisms then apply algebras across all the layers. There is no other mechanism for recursion or looping in the language. All functions written in this style are guaranteed to terminate.

One hindrance to algebraic programming is the need to apply functions for rolling and unrolling signature functors, and to apply $\llbracket - \rrbracket$ to obtain a recursive function from an algebra.

2 Motivating example

3 Syntax

4 Constraint-Generating Typing

5 Subtyping

5.1 Declarative

5.2 Algorithmic

5.3 Equivalence of declarative and algorithmic subtyping

6 Implementation with Chronolog

7 Returning to the Examples

8 Related Work

Turner introduced the term *strong functional programming*, presented several arguments in favor of the approach, and proposed an *elementary* way to realize such a language, in the form of static checks for structural recursion [2]. Mendler proposed a recursor using an abstract type to ensure that recursive calls are permitted only on subdata for an inductive type [1].

9 Conclusion and future work

References

1. N. P. Mendler. Inductive types and type constraints in the second-order lambda calculus. *Annals of Pure and Applied Logic*, 51(1):159 – 172, 1991.
2. D. A. Turner. Elementary Strong Functional Programming. In *Proceedings of the First International Symposium on Functional Programming Languages in Education*, FPLE '95, page 1–13, Berlin, Heidelberg, 1995. Springer-Verlag.